Certifying Graph Isomorphism algorithms with VeriPB

Candidate **Valerio Venanzi**
Advisor **Massimo Lauria**
Computer Science
Sapienza University of Rome

24/01/2024

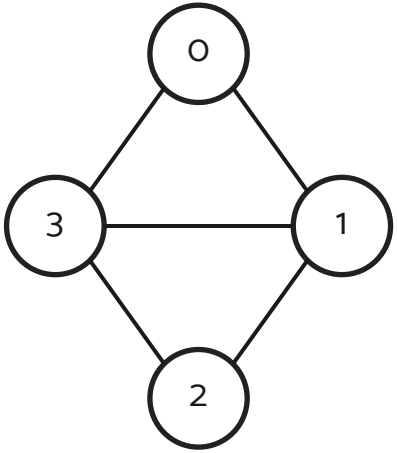
Table of contents

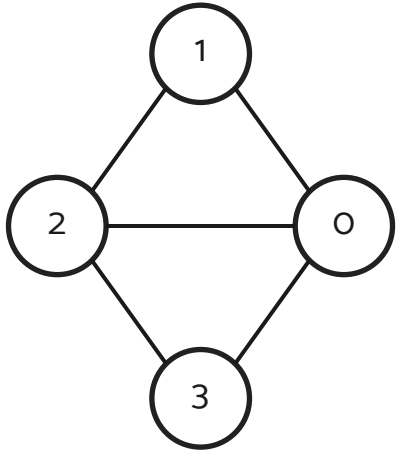
Graph Isomorphism

- Two graphs are said to be isomorphic is there exists a **bijection** between their sets of vertices
- This bijection has to **preserve adjacencies** and **non-adjacencies**:

$$u \sim v \iff f(u) \sim f(v)$$

Graph Isomorphism (example)



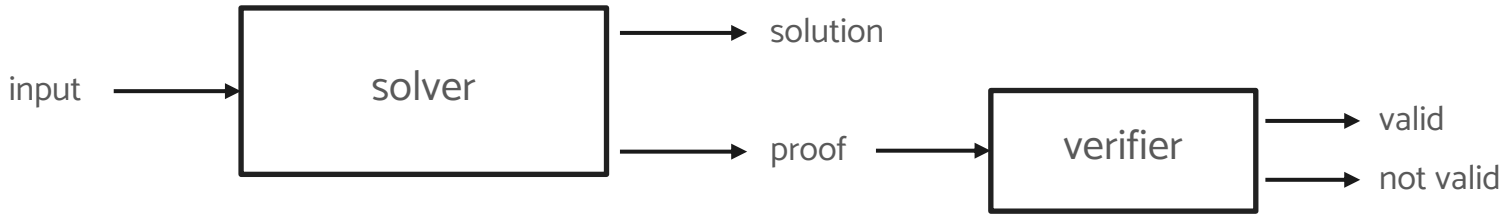


$$f = \begin{cases} 0 \mapsto 1 \\ 3 \mapsto 2 \\ 1 \mapsto 0 \\ 2 \mapsto 3 \end{cases}$$

- These two graphs are isomorphic
- If we rename their vertices in some way we get the same graph

Proof logging

- Proof logging means making the solver also output a proof of the correctness of the solution
- Such program is also said to be **certifying**
- We prove the correctness of the output, NOT of the program
- The proof is given as input to an **independent** program called a **verifier**
- The verifier checks the proof and outputs whether the proof is valid or not



Proof logging

Pros:

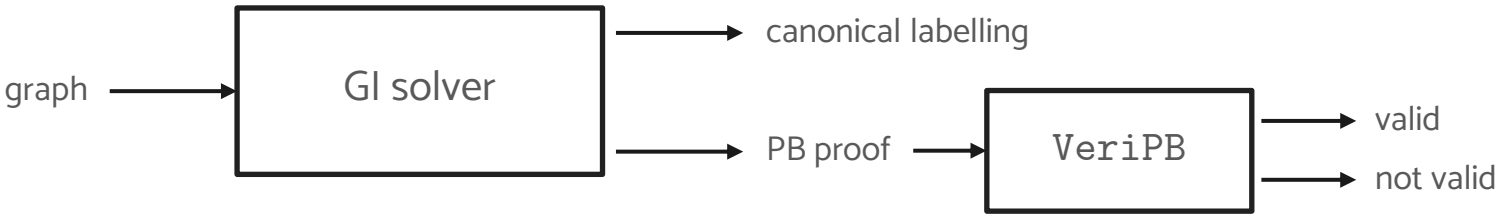
- We are sure that the solution is correct
- Even a correct program can output a wrong solution
- Can assist the programmer in finding bugs during the implementation of the program
- Proofs can be stored and checked at a later time

Cons:

- Slows down the program
- Simple proof format means proofs can get really big
- Checking proof correctness requires time

My work

- **Encode graph isomorphism** problem with integer linear constraints
- Write a simple program for searching **canonical labels**, based on `nauty`[1]
- Make it **certifying**
- **Verify** the proofs
- Run some tests



VeriPB[4]

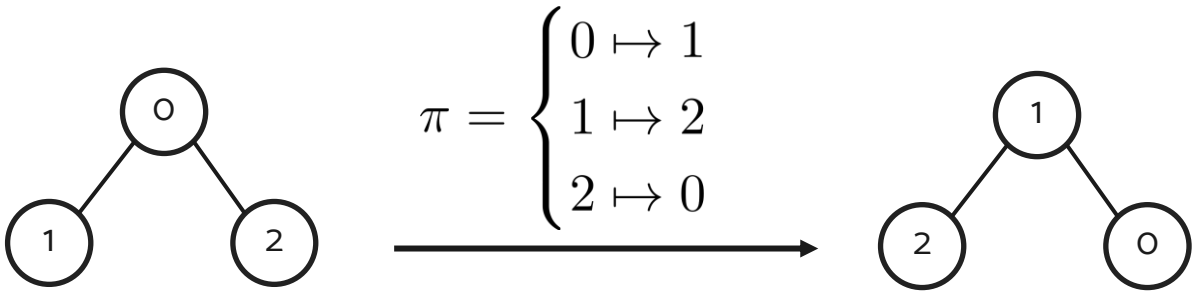
- Verifier that checks **refutations** (proofs of unsatisfiability, i.e. proof that 0 ≥ 1)
- Uses **sound** and **complete** Cutting Planes reasoning over Pseudo-Boolean (PB) constraints
- A PB constraint has the form

$$\sum_i a_i \ell_i \geq A$$

- $a_i \in \mathbb{Z}$
- $A \in \mathbb{Z}$
- ℓ_i is either x_i or $\overline{x}_i$
- $x_i \in \{0, 1\}$ and $x_i + \overline{x}_i = 1$

On the colourings

- We **label vertices** using **numbers** $\{0, 1, \ldots, n-1\}$
- We also **colour vertices** using **numbers** $\{0, 1, \ldots, n-1\}$
- If a colouring π is **discrete** (n distinct colours) then π is a permutation of the vertices
- On each permutation we compute a function φ



Encode graph isomorphism with PB constraints

- We have to encode with PB constraints:
  - Function φ
  - Each colouring must be a permutation
- **Function φ** based on adjacency matrices
- **Colourings** represented as binary permutation matrices

- Two graphs have the same φ value iff they are isomorphic
- So, in practice, we prove that a graph has a certain φ value

Encode graph isomorphism with PB constraints

$$\begin{cases} 1 & \min \varphi \\ 2 & \sum_{i=0}^{n-1} x_{u,i} = 1 \quad (\forall u \in V) \\ 3 & \sum_{u \in V} x_{u,i} = 1 \quad (\forall i \in \{0, \ldots, n-1\}) \end{cases}$$

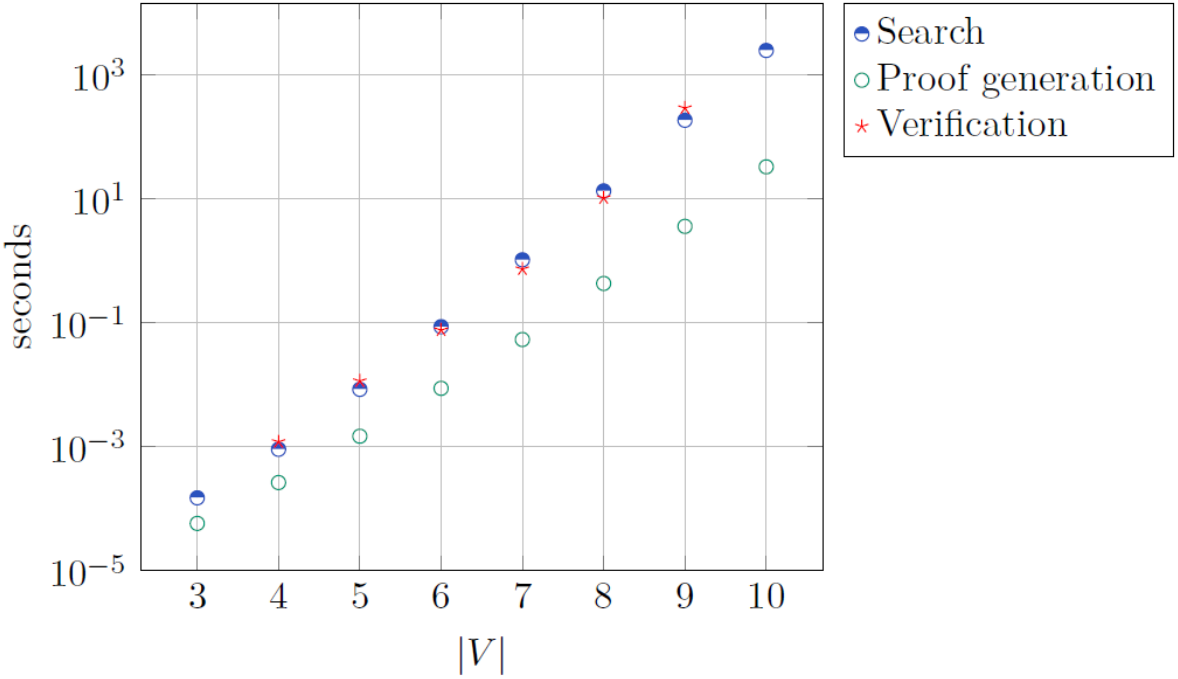
1. Encoding of φ as the minumum of some integer linear program
2. Each vertex must have **exactly** 1 colour
3. Each colour must be given to **exactly** 1 vertex

- There are other constraints needed to correctly encode the problem

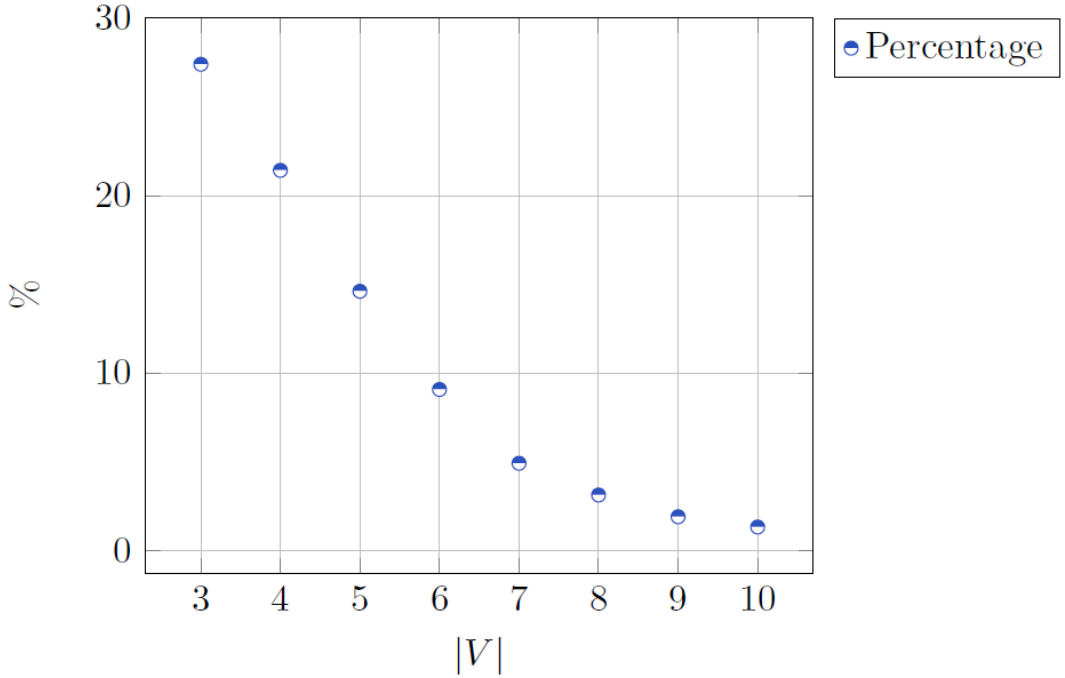
Experimental results

- Tests on **random graphs** with **edge probability 30%**
- The program outputs correct proofs of optimality
- No test has failed
- Interested in:
    - **Search time** and **verification time**
    - **Proof size**

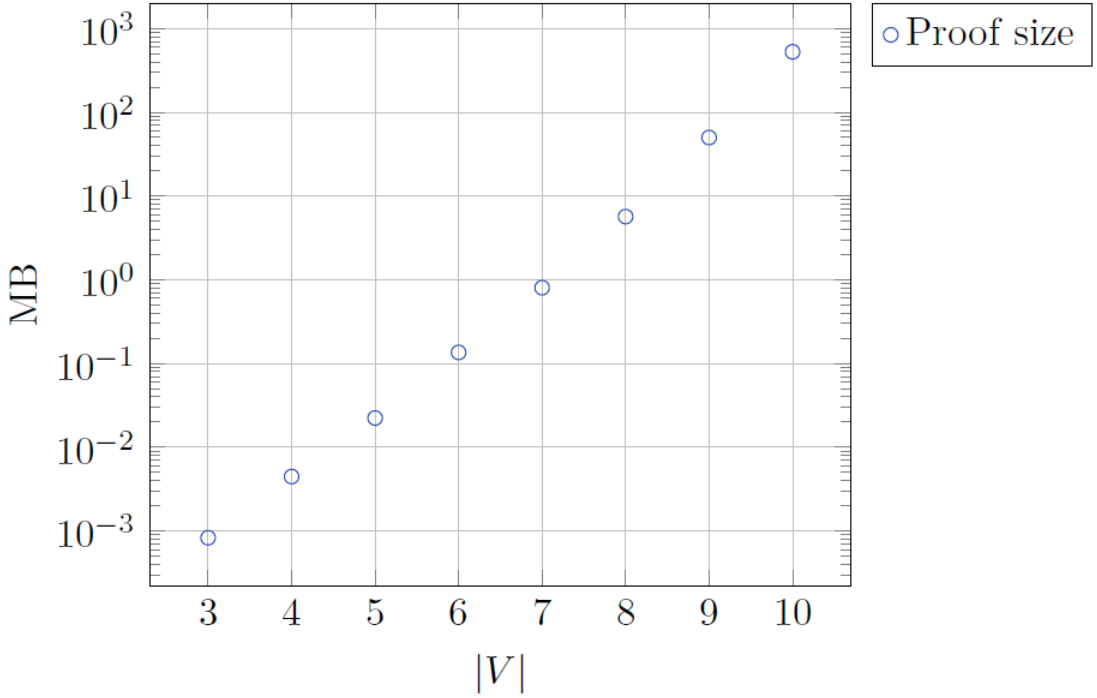
Experimental results (time)



Experimental results (percentage)



Experimental results (size)



Open problems

- Implement proof logging on actual GI solvers (e.g. `nauty`, `Traces`)
  - Many GI solvers are based on the same algorithm
- Encode different φ's
  - φ is any function with some specific properties
- We use a simple proof system to deal with a complex problem
  - Not easy to prove to correctness
  - Proofs get really large

References

- **Papers:**
  - [1] McKay, B. D., & Piperno, A. (2013). Practical graph isomorphism, II.
  - [2] W. Cook, C.R. Coullard, and Gy. Turán. (1987). “On the complexity of cuttingplane proofs”.
  - [3] Marijn J.H. Heule, Warren A. Hunt, and Nathan Wetzler. (2013). “Trimming while checking clausal proofs”
- **Links:**
  - [4] https://github.com/StephanGocht/VeriPB
- **Slides:**
  - https://github.com/pietro-nardelli/sapienza-ppt-template
    [Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)]

Thank you!
Questions?

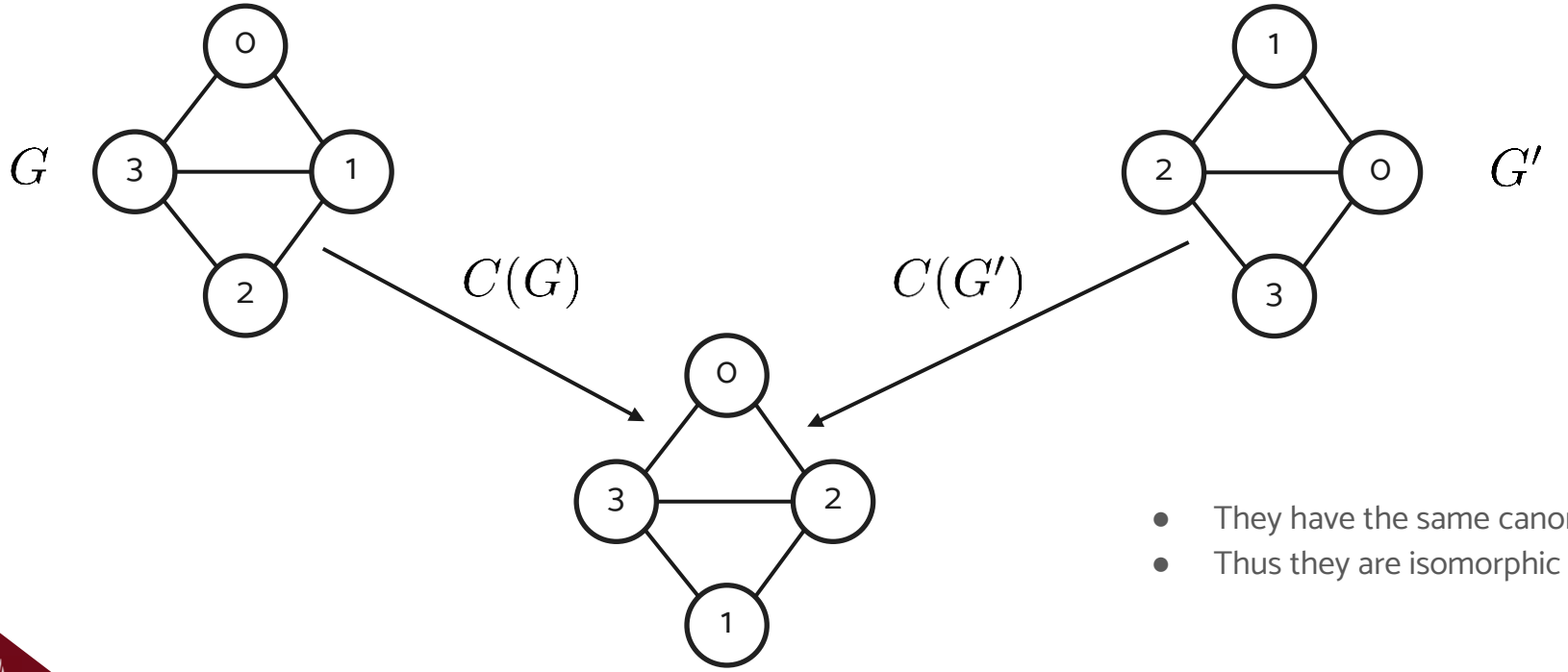
Canonical labelling

- Given a graph G, the canonical graph C(G) is a graph s.t.
  - C(G) is isomorphic to G
  - Any graph G' isomorphic to G has C(G')=C(G)
- To determine if two graphs are isomorphic we reduce the problem to finding their canonical form
- To compute the canonical labelling there is an algorithm based on a search tree due to McKay and Piperno [1]

Canonical labelling

- To find the canonical labelling we **colour** the vertices
- To each colouring we associate a value, computed with a function called φ
- φ has to be **label-invariant**: it depends only on the combinatorial property of the graph and not on the labelling of the graph
- The canonical labelling is the one with the smallest φ

Canonical labelling (example)



- They have the same canonical form
- Thus they are isomorphic

VeriPB proof example

$(C_1) 2x_1 + 3x_2 \geq 4$

$(C_2) 1\overline{x_1} + 1x_2 \geq 1$

`p` $C_1\ C_2\ +\ 2$ `d`

`rup` $2x_1 + 1x_2 \geq 2$

$\vdots$

`rup` $0 \geq 1$

$$\left\lceil \frac{C_1 + C_2}{2} \right\rceil =$$

$$\left\lceil \frac{(2x_1 + 3x_2 \geq 4) + (1\overline{x_1} + 1x_2 \geq 1)}{2} \right\rceil =$$

$$\left\lceil \frac{1x_1 + 4x_2 \geq 5}{2} \right\rceil = 1x_1 + 2x_2 \geq 3$$

Encode GI with PB constraints (linearization of φ)

$$
\begin{cases}
\min \varphi & \\
\sum_{i=0}^{n-1} x_{u,i} = 1 & (\forall u \in V) \\
\sum_{u \in V} x_{u,i} = 1 & (\forall i \in \{0, \dots, n-1\}) \\
\overline{x_{u,i}} + \overline{x_{v,j}} + y_{(u,i),(v,j)} \geq 1 & (\forall \{u,v\} \in E,\ \forall i,j \in \{0, \dots, n-1\}) \\
\overline{y_{(u,i),(v,j)}} + x_{u,i} \geq 1 & (\forall \{u,v\} \in E,\ \forall i,j \in \{0, \dots, n-1\}) \\
\overline{y_{(u,i),(v,j)}} + x_{v,j} \geq 1 & (\forall \{u,v\} \in E,\ \forall i,j \in \{0, \dots, n-1\})
\end{cases}
$$